

HCL

HashiCorp Configuration Language

1 Anatomie d'un main.tf



Le fichier main.tf est le point d'entrée classique d'un projet Terraform.

Structure typique :

1. Bloc terraform (optionnel)
2. Bloc provider
3. Variables (optionnel)
4. Locals (optionnel)
5. Ressources
6. Outputs (optionnel)

Exemple de main.tf :

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}

variable "region" {
  type    = string
  default = "us-east-1"
}

locals {
  name_prefix = "demo"
}

resource "aws_instance" "web" {
  ami          = "ami-12345678"
  instance_type = "t2.micro"
  tags = {
    Name = "${local.name_prefix}-web"
  }
}

output "instance_id" {
  value = aws_instance.web.id
}
```

2 Anatomie d'un bloc



Un bloc est la structure de base en HCL. Il définit un élément (Terraform, provider, ressource, etc.).

```
TYPE "NOM" "ÉTIQUETTE" {
  ARGUMENT = VALEUR

  BLOC imbriqué "NOM" {
    ARGUMENT = VALEUR
  }
}
```

Type de bloc

Nom (optionnel)

Étiquette (identifiant)

Corps du bloc

Exemple :

```
resource "aws_instance" "web" {
  ami          = "ami-12345678"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server"
  }
}
```

3 Types de données



HCL prend en charge plusieurs types de données.

Type	Exemple
string	"hello"
number	42
boolean	true
list (liste)	["a", "b", "c"]
map (carte)	{ key = "value" }
set (ensemble)	toset(["a", "b"])
object (objet)	{ name = "web" port = 80 }
tuple (tuple)	["web", 80, true]



HCL est fortement typé : chaque valeur a un type.

4 Références



Permet de faire référence à d'autres éléments définis dans la configuration.

Types de références :

- Ressources : `aws_instance.web.id`
- Attributs : `aws_instance.web.ami`
- Sorties (outputs) : `output.value`
- Variables : `var.region`
- Locals : `local.name`
- Modules : `module.vpc.id`

Exemple :

```
resource "aws_instance" "web" {
  ami          = "ami-12345678"
  instance_type = "t2.micro"
  subnet_id    = module.vpc.public_subnet_id

  tags = {
    Name        = local.name
    Environment = var.env
  }
}
```



Les références créent des dépendances automatiques entre ressources.

5 Expressions



HCL permet d'écrire des expressions pour calculer des valeurs.

Fonctionnalités :

- Opérateurs arithmétiques : `+`, `-`, `*`, `/`
- Opérateurs de comparaison : `==`, `!=`, `<`, `<=`, `>`, `>=`
- Opérateurs logiques : `&&`, `||`, `!`
- Fonctions intégrées : `length()`, `upper()`, `cidrsumo()`, etc.
- Expressions conditionnelles : `condition ? valeur_true : valeur_false`

Exemples :

```
local.x = var.a * 2 + 1
local.is_prod = var.env == "prod" ? 1 : 0
local.name_prefix = upper(var.app_name)

resource "aws_instance" "web" {
  count = local.is_prod ? 2 : 1
  ami   = var.ami_id
  instance_type = "t2.micro"

  tags = {
    Name = "${local.name_prefix}-${count.index}"
  }
}
```



Les expressions rendent la configuration dynamique et réutilisable.